NED University of Engineering and Technology, Karachi

Computer Science and Information Technology

COMPLEX COMPUTING PROBLEM (CCP)
PROJECT SUBMISSION

Knowledge-Based Systems (CT-359)

Project Title	**Knowledge-Based System for Industrial Equipment Fault Diagnosis**
Submission Date	**18 / October / 2026**

Student Details

Sr. No.	Student Name	Roll No.	Signature
1	Mohammed Sameer Abdo	CT-24302	
2	Mohammed Bilal	CT-24275	
3	Huzaifa Jaffri	CT-24286	
4	Ghazi Hasher Ali	CT-24297	

1. Problem Statement

- **Knowledge representation:** how a Knowledge-Based System (KBS) can hold the equipment, component, symptom, fault, cause and maintenance-action knowledge that maintenance staff usually carry in their heads or in scattered manuals.
- **Problem analysis:** the fault-diagnosis problem is under-specified, so the missing requirements, stakeholder needs, relationships among knowledge elements and constraints must be identified.
- **Method evaluation:** alternative representation and inference methods must be compared, and the chosen one justified on accuracy, explainability, safety, cost and system requirements.

2. Problem Description / Project Context

A manufacturing plant usually runs many kinds of machines at once: pumps moving fluids, electric motors driving production lines, compressors supplying air, and gearboxes transmitting power. When one of them starts behaving abnormally, the maintenance team must decide quickly what is wrong and what to do about it. This is harder than it sounds. The knowledge needed for that decision is spread across maintenance manuals, operator notes, past repair records, the experience of senior technicians, and live sensor readings such as vibration, temperature and pressure. None of these sources talks to the others, and much of the know-how is never written down at all [1][2].

The situation is made more difficult by the symptoms themselves. A single symptom, such as high vibration, can point to several faults, while one fault can show up through several symptoms at once. Operators often describe problems vaguely ("it sounds strange"), sensors can fail or be missing, and sources sometimes disagree. The people involved also want different things. Safety officers want risky machines stopped immediately, production managers want the line to keep running, and the budget holder wants the cheapest fix. The organization therefore needs a KBS that can represent this mixed knowledge, reason about incomplete and conflicting evidence, rank the most likely faults, recommend a sensible maintenance action, and explain how it reached its conclusion.

3. Problem Analysis

3.1 Problem Understanding

At its core, this is diagnosis under uncertainty. The maintenance person starts with observations and must work backwards to the underlying fault, then choose the right action. Doing this by hand is slow and depends heavily on who is on shift: an inexperienced operator may miss a serious fault, and even an expert can overlook something when symptoms overlap.

We identified four main difficulties. First, knowledge is scattered across formats and is partly informal. Second, the same symptom can belong to many faults, so a one-to-one lookup does not work. Third, evidence is rarely complete or fully reliable. Fourth, a diagnosis alone is not enough: the system must also recommend an action that respects safety, cost and downtime. Together these rule out a fixed, hard-coded solution and point towards a knowledge-based approach in which knowledge is stored explicitly and the reasoning can be inspected.

3.2 Stakeholders and Their Requirements

We identified five groups who deal with the system, each needing something different from it.

i. **Machine operators:** They notice problems first. They need a simple way to enter what they see and hear, and a quick answer in plain language.
ii. **Maintenance engineers and technicians:** They do the repairs. They need a ranked list of likely faults, probable causes and a clear action, with the reasoning shown so they can trust or challenge the result.
iii. **Safety officers:** They prevent injuries and accidents. They need dangerous conditions flagged immediately, with a clear stop recommendation when risk is high.
iv. **Production managers:** They care about output. They need the downtime of each recommended action to be visible so they can plan around it.
v. **Maintenance and finance managers:** They control the budget. They need the estimated cost of each action and want to avoid unnecessary part replacement.

3.3 Constraints and Conflicting Requirements

Several constraints shape the solution. Safety is the strictest: faults such as severe overheating or a failing pressure component cannot be allowed to continue regardless of the cost of stopping. Downtime and cost are softer but matter greatly in practice. Actions also have different urgency, so the system must separate what needs immediate action from what can be scheduled or monitored. On the practical side, data is limited, real experts are not always available, and the project had a short timeframe, which limits how much equipment and knowledge could be covered.

The stakeholder requirements conflict. Safety wants a machine stopped at the first sign of danger, while production wants it running. The cheapest repair is often not the fastest, and the fastest is often not the safest. Our decision is that safety always takes priority, and that cost, downtime and production impact are used to rank the remaining options once safety is satisfied. Section 5.4 explains how this decision is implemented and where the implementation falls short.

3.4 Root Cause and Requirement Identification

The brief states the problem in general terms, so part of the analysis was to identify what is missing and what the root causes are.

The main root cause of the diagnosis difficulty is that equipment knowledge is fragmented and mostly implicit. It is spread over manuals, records and people's memories, and is not stored in a form a computer can reason with [1]. A second root cause is the many-to-many relationship between symptoms and faults, which makes simple rule lookups unreliable. A third is that uncertainty is not handled explicitly: decisions are often made on judgement when information is missing.

The requirements the brief leaves unclear, which we therefore defined ourselves, are:

- **Equipment types to cover.** We selected four: a centrifugal pump, a three-phase induction motor, an industrial air compressor and an industrial gearbox.
- **Inputs.** Observable symptoms such as noise, vibration, overheating, leakage and loss of flow, and optional telemetry readings (air and process temperature, rotational speed, torque and tool or component wear).
- **Treatment of missing or doubtful evidence.** Certainty factors (CF), so that missing or doubtful evidence lowers confidence rather than being ignored.
- **Conflicting evidence and goals.** Rule combination for agreeing evidence, and a safety-first priority for conflicting goals.
- **Explanation.** The list of rules that fired and the evidence behind each conclusion, presented as a six-step trace.

3.5 Assumptions

To keep the project achievable and clearly defined, we made the following assumptions. Each is justified where it is stated.

1) The system covers four equipment types. This keeps the knowledge base manageable within the project timeframe and matches the four machine classes named in the course brief.

2) Symptom and sensor values are entered by the user. No live sensor connection exists, because the project has no plant data stream; telemetry is entered manually in the interface.

3) In one diagnosis session, one main fault is assumed to be present, although several candidate faults can be ranked. This is a simplification: real faults can be concurrent, and Section 9.2 discusses the effect.

4) The knowledge base is built from public sources such as maintenance guides, published research and open datasets [1]–[3], [6]–[27]. It is not taken from a specific plant, so thresholds are reference values and not plant-calibrated.

5) Certainty factors are assigned from the knowledge sources and our judgement, since no expert panel was available to calibrate them. In the implemented code every rule uses the same base value of 0.80, so the CF values carry no per-rule information (Section 9.2).

6) The system is a decision-support tool. It recommends and explains, but final decisions stay with the human maintenance staff, because it has not been validated against plant outcomes.

7) Safety takes priority over cost and downtime whenever they conflict. This is a design decision, justified by the safety-critical nature of the equipment and by the safety officers' requirements.

4. Knowledge Representation and KBS Design

4.1 Selected Knowledge Representation Formalism

We use hybrid representation. Frames (Minsky, 1974 [5]) describe the structure of each machine: its name, its components and its observable symptoms. Production rules (IF-THEN) map a symptom to a fault cause, a corrective remedy and the affected component, and carry a base certainty factor and a safety priority. A separate set of quantitative rules covers telemetry thresholds.

Table 1 compares the candidate formalisms against the project's requirements. We chose production rules with forward chaining for three reasons. The fault is unknown and the evidence arrives as symptoms, so reasoning should move forward from data to hypotheses. Rules are the most transparent formalism for explanation, which the brief requires. And the available data is a set of symptom-cause-remedy records, which map directly onto rules. Ontologies and knowledge graphs are powerful for linked data [1][2][3], but they add schema and graph-storage effort that this single-fault, four-machine prototype does not need. A learned classifier would need labelled fault records, which we do not have, and would be harder to explain.

Method	Fit to problem	Explainability	Data needed	Decision
Production rules + forward chaining	Good: data-driven, evidence arrives first	High: each rule and step is visible	Symptom-cause-remedy records (available)	Chosen
Production rules + backward chaining	Weaker: starts from a hypothesis, but the fault is unknown	High	Same as above	Rejected for the main path
Ontology / knowledge graph reasoning	Good for linked relations; more schema effort	Medium to high	Entities and typed relations	Not needed for this scope; noted as future work
Bayesian network	Handles uncertainty well	Medium	Conditional probabilities (not available)	Rejected: no probability data
Machine learning classifier	Possible with data	Low: black box	Labelled fault records (not available)	Rejected: no labelled data; hard to explain

4.2 Knowledge Base / Ontology / Rule Base

The knowledge base holds 299 production rules, 86 unique symptoms and 158 distinct components across four frames. Each rule is loaded from one row of a CSV file and given a generated identifier (for example MOTOR_R001). Table 2 summarises the knowledge base, including the distribution of safety priorities, which is assigned by a keyword heuristic on the cause and remedy text.

Knowledge-base contents by machine (measured from the implemented code)

Machine	Rules	Symptoms	Components	Critical	High	Medium	Low
Three-phase induction motor	91	31	41	1	23	12	55
Centrifugal pump	59	12	31	0	14	24	21
Industrial gearbox	76	27	35	0	31	23	22
Industrial air compressor	73	16	51	0	38	9	26
Total	299	86	158	1	106	68	124

A representative rule, taken unchanged from the motor source file, is:

```
IF   symptom    = "Motor does not start on load."
THEN cause      = "No supply voltage"
     remedy     = "Check voltage at Motor Terminals."
     component  = "Power supply"
     base CF    = 0.80      priority = MEDIUM   (keyword: 'check')
```

The rule content is taken from four CSV files whose source columns cite the guides listed in Section 6. The certainty values are not tuned per rule: the same base CF of 0.80 is assigned to every rule (Section 9.2).

4.3 Inference Mechanism

The inference engine (inference_engine.py) performs forward chaining with a match-resolve-act cycle:

- **Match.** The observed symptoms are placed in working memory. Each rule's symptom is matched against them by normalised text equality, falling back to substring containment for compound phrases.
- **Aggregate.** Fired rules are grouped by cause name, so that two symptoms pointing to the same cause are combined into one hypothesis.
- **Combine certainty.** For each hypothesis, the unique supporting symptoms are combined with the Shortliffe and Buchanan MYCIN formula [4]: CF_combined = CF1 + CF2 × (1 − CF1), applied iteratively. Two symptoms with CF 0.80 give 0.96; three give 0.992. The result is rounded to two decimals.
- **Corroborate.** Telemetry readings are checked against the sensor rules and can raise the certainty of a related hypothesis (Section 7.4).
- **Resolve.** Hypotheses are ordered by the conflict-resolution strategy in Section 5.4.

For a single observed symptom, the certainty of each fired hypothesis is the base 0.80. Certainty is never 1.0: the cap is 0.99, reflecting that no empirical rule is absolute.

4.4 Conflict Resolution Strategy

When several hypotheses fire, the engine orders them by three keys in turn: the safety-priority weight (CRITICAL 4, HIGH 3, MEDIUM 2, LOW 1), then certainty, then the number of supporting symptoms. This implements the safety-first decision from Section 4.3. Conflicting evidence is handled in two ways. Agreeing evidence is combined so that corroborated hypotheses rise. Competing hypotheses are kept as ranked alternatives rather than discarded, which suits the rule that the final decision stays with the technician.

There is a gap between design and implementation, which we found during testing. The API layer (app.py) re-sorts the engine's output by certainty first and priority second. The effect is that a LOW-priority hypothesis with a higher certainty is shown above a HIGH-priority hypothesis with lower certainty. Table 6 (Section 8.4) reproduces this. The safety-first order is therefore correct inside the engine but not in the output the user sees. Costs and downtime are not used in ranking at all; they appear only as remedy text. Section 9.2 discusses both points.

4.5 Explanation Facility

Each diagnosis carries a six-step reasoning trace, which answers the question "how was this conclusion reached?" The steps are: (1) the observed symptoms; (2) the rules that fired, in IF-THEN form; (3) the recommended remedy; (4) the affected component; (5) the certainty calculation, with the formula used; and (6) the safety priority and the keyword that justified it. The trace below is produced by the engine for the pump case in Section 8.2.

```
Step 1: Symptoms "Bearings run hot", "Pump is noisy or vibrates excessively" were observed by the
user
Step 2: Rules PUMP_R029, PUMP_R047 matched — IF symptoms observed THEN cause is "Worn bearings"
Step 3: Recommended action: "Inspect and replace the bearings"
Step 4: Affected component: "Bearing"
Step 5: Certainty calculated as 0.96 (multiple symptom boost from 2 symptoms)
Step 6: Safety priority: HIGH (explicit priority defined in rule )
```

The trace is the main transparency feature. Unlike a black-box model, every number can be traced back to a rule ID and a source row.

4.6 System Architecture

The system follows Feigenbaum's separation of knowledge base, inference engine and user interface. Figure 1 shows the components and the data flow.

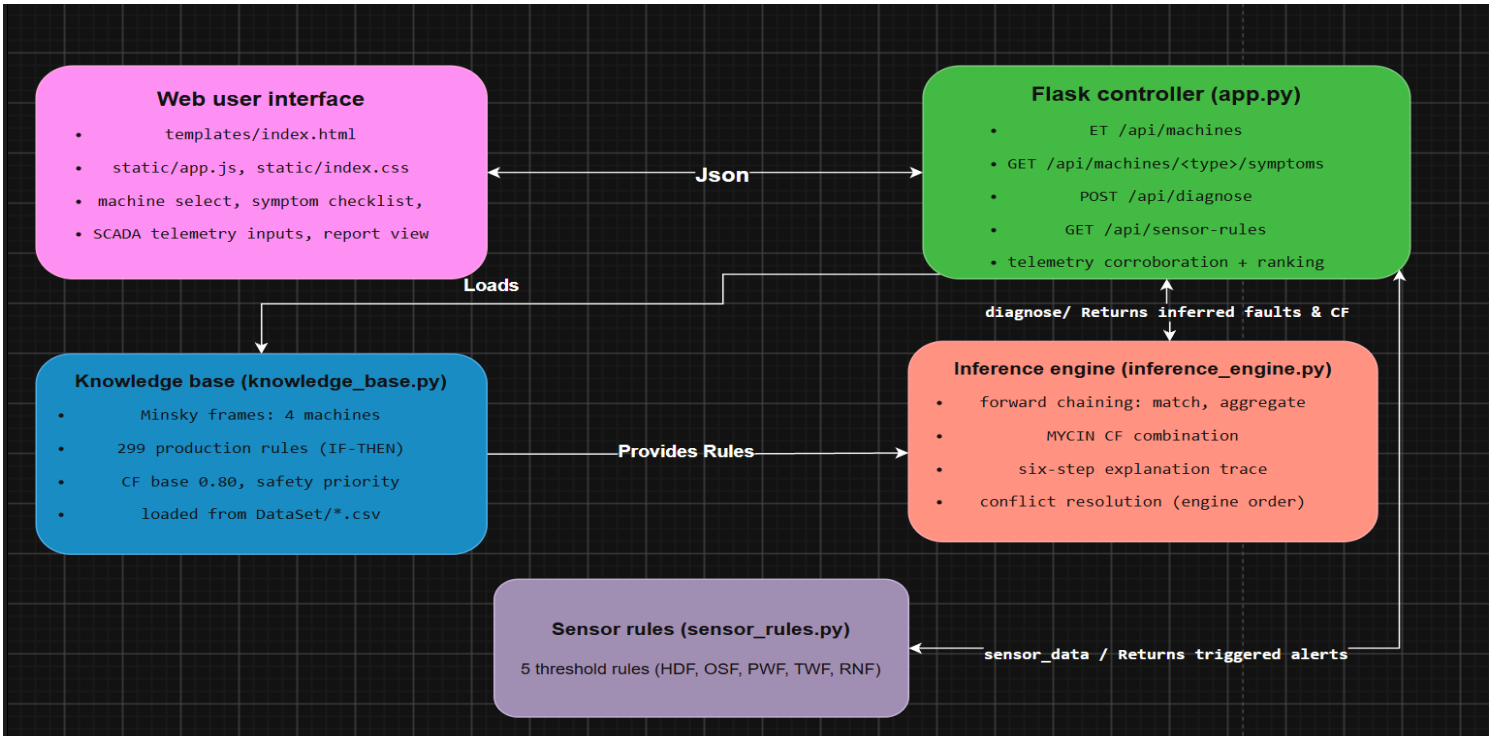


Figure 1. Architecture of the KBS (Flask backend, rule-based knowledge base, forward-chaining engine, sensor rules, web interface)

5. Dataset / Knowledge Sources

Table 3 lists every knowledge source and dataset used. The four machine files are knowledge bases built from maintenance and troubleshooting guides; the AI4I 2020 file is a telemetry dataset used to test the sensor rules. The full source citation for each rule is kept in the Source column of its CSV file and is reproduced in Section 10.

Table 3. Dataset and knowledge sources

Source / Dataset	Type	Size / No. of records	Purpose	Citation / URL
motor_faults.csv	Expert troubleshooting knowledge (text)	91 rules, 31 symptoms, 41 components	Symptom-cause-remedy rules for induction motors	[7], [8]
pump_faults.csv	Expert troubleshooting knowledge (text)	59 rules, 12 symptoms, 31 components	Cavitation, suction loss, impeller and seal faults	[9]–[17]
gearbox_faults.csv	Expert troubleshooting knowledge (text)	76 rules, 27 symptoms, 35 components	Tooth pitting, lubrication, misalignment, overheating	[18]–[22]
compressor_faults.csv	Expert troubleshooting knowledge (text)	73 rules, 16 symptoms, 51 components	Valve, oil, cooling and control faults	[23]–[27]
ai4i2020.csv	Telemetry dataset (synthetic, UCI)	10,000 records; 339 machine-failure records	Testing the telemetry threshold rules (Section 8.3)	[6]
Research and theory	Review papers; original method papers	3 review papers; 2 method papers	Background on KG-based KBS; CF and frame theory	[1]–[5]

6. Implementation

6.1 Software / Tools / Libraries

- Python 3 (standard library for CSV, regular expressions, dataclasses and logging).
- Flask, the only third-party package, for the web server and REST API.
- HTML, CSS and vanilla JavaScript for the dashboard. No database and no machine-learning library are used.
- Windows launcher run.bat, which starts app.py on port 5000. The system runs locally in a browser at http://localhost:5000.
- matplotlib was used only to draw Figure 1 for this report; it is not part of the application.

6.2 Modules and Components

File	Role	Main content
knowledge_base.py	Knowledge base	Parses the four CSV files (quoted multi-line fields, BOM handling). Builds Rule and Frame objects, generates rule IDs, and assigns safety priority by keyword heuristic.
inference_engine.py	Inference engine	Forward chaining: match, aggregate, MYCIN certainty combination, conflict resolution, six-step explanation trace.
sensor_rules.py	Telemetry rules	Five threshold rules (HDF, OSF, PWF, TWF, RNF) with a failure-type to machine mapping.
app.py	Controller / REST API	Endpoints, input validation, telemetry corroboration, direct telemetry diagnoses, final ranking.
templates/index.html, static/app.js, static/index.css	User interface	Machine selection, symptom checklist, telemetry inputs with presets, ranked result cards, printable report.

6.3 Code Structure

The project folder is organised as follows. The DataSet folder holds the four machine CSV files and the AI4I 2020 telemetry file. The static and templates folders hold the interface. The reference papers are kept separately in the Refrences folder.

```
Knowledge-Based System for Industrial Equipment Fault Diagnosis/
├── app.py                  Flask controller and REST API
├── knowledge_base.py       Frames, rules, CSV loader
├── inference_engine.py     Forward chaining, CF algebra, explanations
├── sensor_rules.py         Telemetry threshold rules
├── templates/index.html    Dashboard
├── static/app.js           Client logic
├── static/index.css        Styling and print layout
├── DataSet/                motor, pump, gearbox, compressor CSVs; ai4i2020.csv
├── run.bat                 Windows launcher
└── CCP (Report + instruction)/  assignment brief and report template
```

Two code excerpts carry the core reasoning. The first is the MYCIN combination in inference_engine.py, which is the basis of all multi-symptom certainty values:

```
combined_cf = base_cfs[0]
for next_cf in base_cfs[1:]:
    combined cf = combined cf + next cf * (1.0 - combined cf)
```

The second is the telemetry corroboration step in app.py. A sensor rule that matches a related hypothesis raises its certainty:

```
combined_cf = old_cf + (1.0 - old_cf) * (s_rule.certainty * 0.5)
new cf = min(0.99, round(combined cf, 4))
```

The REST endpoints are: GET /api/machines, GET /api/machines/<type>/symptoms, GET /api/machines/<type>/components, POST /api/diagnose and GET /api/sensor-rules.

6.4 User Inputs and Outputs

Inputs. The operator selects one of the four machines, ticks one or more observed symptoms from a checklist, and may optionally enter telemetry: air temperature (K), process temperature (K), rotational speed (rpm), torque (Nm) and tool or component wear (min). Preset buttons fill the telemetry fields with example values.

Processing. The request is checked (a machine and at least one symptom or reading are required), forward chaining runs on the symptoms, the telemetry rules are evaluated and filtered to the chosen machine, and any triggered rule raises a related hypothesis or creates a direct telemetry diagnosis.

Outputs. A ranked list of diagnoses. Each entry gives the fault, the affected component, the remedy, the certainty, the safety priority, the matched symptoms, the rule IDs, a sensor-corroboration flag and the reasoning trace. An example of a single result as returned by the API is:


```
{
  "cause": "Worn bearings",
  "remedy": "Inspect and replace the bearings",
  "component": "Bearing",
  "certainty": 0.96,
  "safety_priority": "HIGH",
  "matched_symptoms": ["Bearings run hot", "Pump is noisy or vibrates excessively"],
  "matched_rules": ["PUMP_R029", "PUMP_R047"],
  "sensor_supported": false
}
```

7. Testing and Results

Testing covered three things: functional behaviour of the four test scenarios, the arithmetic of certainty combination and sensor boosting, and measured accuracy of the telemetry rules against labelled AI4I 2020

records. All results below were produced by running the implemented code on the project files. No labelled symptom-level fault records exist, so the accuracy of the symptom rules themselves could not be measured (Section 9.2).

7.1 Functional Test Cases

Table 4. Functional test results

Test	Input	Observed result	Status
T1: single symptom	Pump: "Pump does not deliver any liquid"	6 hypotheses returned, all at CF 0.80. The top result is "Wrong direction of rotation" (HIGH). "Pump not primed" is ranked 5 of 6 at LOW priority.	Pass for functionality. The not-primed cause is not the top result.
T2: multi-symptom CF boost	Motor: "Motor does not start on load." + "Overheating"	11 hypotheses, each supported by one symptom at CF 0.80. The two symptoms share no cause, so no combination occurs.	Not reproduced. The pair was not a valid boost test.
T2 (corrected): boost	Motor: "Motors runs and then stops" + "Motor overheates while running on load."	"Overload" with 2 matched symptoms, CF 0.96, from rules MOTOR_R016, MOTOR_R044.	Pass. CF 0.80 + 0.80 × 0.20 = 0.96.
T2 (corrected): boost, pump	Pump: "Bearings run hot" + "Pump is noisy or vibrates excessively"	"Worn bearings" with 2 matched symptoms, CF 0.96.	Pass.
T3: telemetry corroboration	Compressor: "Overheating and thermal shutdown" + HDF telemetry (Air 296.0 K, Process 314.5 K)	No corroboration and no telemetry alert. The HDF rule fires for these readings, but the compressor is not in its machine mapping, so it is discarded. Result certainty 0.80.	Fail. Compressor cannot be corroborated by HDF.
T3 (control): telemetry corroboration	Motor: "Overheating" + the same HDF telemetry	Corroborated. The top hypothesis rises from CF 0.80 to 0.878 with the sensor-corroborated flag set.	Pass for the boost path. The HDF condition itself is wrong (Section 8.3).
T4: print report	Any completed diagnosis; Print Report button	Not run in this testing round. Requires a browser session.	Not verified.

7.2 Certainty Combination Verification

The six-step trace for the pump pair in T2 is shown in Section 5.5. Its certainty step matches the MYCIN formula exactly: $0.80 + 0.80 \times (1 - 0.80) = 0.96$. The same check was run for the motor overload pair, which also gives 0.96.

7.3 Telemetry Rule Evaluation on AI4I 2020

Each of the 10,000 AI4I 2020 records was passed through the five sensor rules and compared with the dataset's own failure labels. A rule is counted as a true positive when it fires on a record labelled with that failure mode. Table 5 gives the results. Because one record can carry several failure labels, the per-mode counts do not sum to the 339 machine-failure records.

Table 5. Sensor rule performance against AI4I 2020 labels

Failure mode	Labelled records	True positives	False positives	False negatives	Precision	Recall
HDF	115	0	4403	115	0.000	0.000
OSF	98	18	0	80	1.000	0.184
PWF	95	12	0	83	1.000	0.126
TWF	46	43	747	3	0.054	0.935
RNF	19	0	4	19	0.000	0.000
Any rule vs machine failure	339	184	4663	155	0.038	0.543

The HDF result is the clearest failure. The dataset's HDF records all have a temperature difference (process minus air) between 7.6 K and 8.6 K, and rotational speeds between 1212 and 1379 rpm. All 115 of the 115 HDF records satisfy both "difference below 8.6 K" and "speed below 1380 rpm". The implemented rule instead requires the difference to be above 8.6 K, so it is inverted. It fires on 4403 records and catches none of the true HDF failures.

The OSF and PWF rules are precise but miss most failures, so they are safe to act on when they fire but weak as detectors. The TWF rule catches most true failures but also fires on many healthy records, because it flags a wear window without considering the other conditions. The RNF rule never fires correctly. These results do not validate the sensor rules, and they should not be read as evidence that the telemetry path is reliable.

7.4 Ranking Verification

Table 6 shows the API output for the pump pair in T2 (Section 8.1). The engine itself would order these by safety priority first. The table shows the order that the user actually receives.

Table 6. API output order for the pump pair (certainty is sorted before safety priority)

Position	Cause	Certainty	Safety priority
1	Worn bearings	0.96	HIGH
2	Improper alignment of pump and driver	0.96	LOW
3	Broken or bent impeller or impeller shaft	0.80	HIGH
4	Improper lubrication (insufficient, wrong or contaminated lubricant)	0.80	MEDIUM

In position 2, a LOW-priority cause (improper alignment, CF 0.96) appears above a HIGH-priority cause (broken or bent impeller, CF 0.80). This contradicts the safety-first decision in Section 4.3.

7.5 Defects Found During Testing

Testing found the following defects. They are listed here so that the evaluation in Section 9 can be read against them. Each is reproducible with the files in the submitted project.

ID	Finding	Evidence	Suggested fix
D1	HDF telemetry condition is inverted	HDF records have $\Delta T < 8.6$ K and speed < 1380 rpm; the rule tests $\Delta T > 8.6$ K. Recall 0.000.	Change the condition to $\Delta T < 8.6$ K and speed < 1380 rpm; re-run Table 5.
D2	Compressor cannot receive HDF corroboration	HDF machine mapping excludes compressor, but the default components table includes it.	Add compressor to the HDF mapping, or remove its entry from the defaults.
D3	Output ranks certainty before safety	app.py sorts by (certainty, priority); engine sorts by (priority, certainty, evidence).	Sort by priority first in app.py, as the engine does.
D4	Boost magnitude differs from design	The design text describes a +0.10 boost. The code applies $CF + (1 - CF) \times 0.5 \times CF_rule$, giving 0.80 to 0.878 for HDF.	Use one boost formula and document it, or change the design text to match.
D5	Telemetry keys differ between engine and API	Engine checks keys such as 'temperature'; the interface sends 'process_temp'. The engine's own boost path is never triggered from the interface.	Use one telemetry schema across all modules.
D6	Cost and downtime not modelled	No cost or downtime fields exist in the rule base or in the ranking.	Add cost and downtime attributes to rules and use them as tie-breakers after safety.

8. Evaluation, Limitations and Conclusion

8.1 Evaluation of the Proposed Approach

Against the course learning outcomes, the project demonstrates CLO 1 and CLO 2 reasonably well. The knowledge representation is explicit and inspectable, the knowledge base is substantial (299 rules, 86 symptoms, 158 components), and the stakeholder, constraint and requirement analysis in Section 4 is complete. The rule-based approach was selected over alternatives in Table 1 with stated reasons.

CLO 3 is partly met. The comparison of methods is reasoned, and the certainty arithmetic and explanation trace work as designed. However, the evaluation exposed defects that the implementation must fix before the system can be trusted in use: the inverted HDF telemetry rule, the ranking inconsistency between engine and API, and the absence of cost and downtime modelling. A critical evaluator would expect these to be reported, and we report them here rather than leave them out.

Table 7 relates the work to the five rubric criteria, so the evidence is easy to locate.

Rubric criterion	Where the evidence is	Current position
1. Understanding the problem	Section 2, Section 4 (stakeholders, constraints, requirements, assumptions)	Strong: requirements and conflicts are identified and defended.
2. Research and knowledge acquisition	Section 6, Section 10; 27 cited sources; AI4I evaluation	Strong: knowledge drawn from 21 field sources plus a telemetry dataset.
3. Proposed approach and representation	Section 5.1 and Table 1; 5.2 to 5.6	Strong: hybrid frames and rules, with a justified method comparison.
4. Implementation and code	Section 7; Section 8 test results; Section 8.5 defects	Functional, with defects D1 to D6 that must be fixed.
5. Report, presentation, documentation	This report, following the template	Complete against the template; presentation to be prepared separately.

8.2 Limitations

- **No accuracy measurement for symptom rules.** No labelled symptom-level fault records exist, so the diagnostic accuracy of the rule base is unknown. Only functional behaviour and the arithmetic of certainty combination were verified.
- **Knowledge provenance.** The knowledge comes from vendor and web troubleshooting guides, not from a specific plant or from the IEC and ISO standards named in the project notes. The source columns of the CSV files do not cite those standards as sources.
- **Uniform certainty factors.** Every rule uses the same base CF of 0.80. The CF values therefore do not express relative confidence between rules, and the certainty values of different hypotheses are not comparable in a meaningful way.
- **Single-fault assumption.** The system assumes one main fault. Concurrent faults, which are common in real machines, are not modelled.
- **Stakeholder constraints partly unmodelled.** Cost and downtime are not represented (defect D6). Ranking uses safety priority, certainty and evidence count only.
- **Keyword-based safety and corroboration.** Safety priority is assigned by keyword matching on the text of the cause and remedy. It is a heuristic, not a validated safety classification. Telemetry corroboration uses keyword matching between the rule text and the hypothesis text, so it can match on a shared word rather than a physical link.
- **Telemetry rules are weak.** The sensor rules are not validated (Table 5), and the HDF rule is inverted (D1). Telemetry should not be relied on until these are corrected.
- **Deployment settings.** The web server runs in debug mode and allows cross-origin requests from any origin. These settings are acceptable for a local prototype but would need to change before any shared deployment.
- **Matching.** Symptom matching falls back to substring matching, so a short symptom can match longer phrases unintentionally.

8.3 Future Improvements

- Correct the HDF condition and the compressor mapping (D1, D2), and re-run Table 5 to confirm the change.
- Make the ranking consistent by sorting on safety priority first in the API, as the engine does (D3).
- Calibrate certainty factors per rule with input from maintenance engineers, and record the basis for each value.
- Add cost and downtime attributes to rules, and use them as tie-breakers once safety is satisfied (D6).

- Collect a labelled set of symptom-fault cases from the maintenance records and measure accuracy, precision and recall for the symptom rules.
- Support concurrent faults by ranking combinations of hypotheses rather than one fault per session.
- Add an automated test suite covering each defect, so that fixes can be verified.
- Connect the telemetry input to a SCADA or historian feed instead of manual entry.

8.4 Conclusion

This project built a rule-based, forward-chaining knowledge-based system for four types of industrial equipment. It represents equipment structure as frames, fault knowledge as 299 production rules drawn from maintenance sources, and uncertainty with MYCIN certainty factors. The system produces ranked diagnoses with recommended remedies and a transparent reasoning trace that a technician can audit.

Testing shows that the core reasoning works: the rules load, the certainty arithmetic is correct, and the explanation trace is generated. It also shows that important parts do not yet meet the design. The telemetry rules perform poorly, the ranked output does not follow the safety-first rule, and cost and downtime are not modelled. These findings are reported openly. The system is a sound base for a decision-support tool, but it needs the fixes in Section 8.5 and a labelled evaluation before it could be used in practice.

2. References

[1] R. Kang, J. Zhu, S. Green and I. Brilakis, "Knowledge graphs for operational decision-making in industrial maintenance: A systematic review," Expert Systems with Applications, vol. 299, p. 130172, 2026. https://doi.org/10.1016/j.eswa.2025.130172

[2] R. Liu, R. Fu, K. Xu, X. Shi and X. Ren, "A review of knowledge graph-based reasoning technology in the operation of power systems," Applied Sciences, vol. 13, p. 4357, 2023. https://doi.org/10.3390/app13074357

[3] Q. Chen, Q. Li, J. Wu, C. Mao, G. Peng and D. Wang, "Application of knowledge graph in power system fault diagnosis and disposal: A critical review and perspectives," Frontiers in Energy Research, vol. 10, p. 988280, 2022. https://doi.org/10.3389/fenrg.2022.988280

[4] E. H. Shortliffe and B. G. Buchanan, "A model of inexact reasoning in medicine," Mathematical Biosciences, vol. 23, no. 3–4, pp. 351–379, 1975. (MYCIN certainty factors.)

[5] M. Minsky, "A framework for representing knowledge," MIT AI Laboratory Memo 306, 1974. (Frame theory.)

[6] S. Matzka, "AI4I 2020 Predictive Maintenance Dataset," UCI Machine Learning Repository, 2020. Dataset supplied as DataSet/ai4i2020.csv.

[7] "Fault, Causes and Remedies for: Induction Motor," troubleshooting chart (publisher not stated; undated), http://2.imimg.com/data2/OT/VK/MY-/untitled-2.pdf

[8] Malloy Electric, Electric Motor Troubleshooting: A Complete Field Guide for Low Voltage NEMA and IEC Motors, https://www.malloyelectric.com/motor-troubleshooting (accessed Oct 2026)

[9] PSG Dover, Centrifugal Pump Troubleshooting Guide, 2022, https://www.psgdover.com/blackmer/news/article/2022/05/04/centrifugal-pump-troubleshooting-guide

[10] Grundfos, Centrifugal Pump Installation and Operating Instructions, n.d., https://api.grundfos.com/literature/Grundfosliterature-6142526.pdf

[11] Shubham Industries, Centrifugal Pump Troubleshooting Guide, n.d., https://www.shubhampump.com/troubleshooting

[12] General Pump & Machinery, Centrifugal Pump Troubleshooting, n.d., https://www.generalpumpandmachinery.com/pdf/Troubleshooting%20Centrifugal%20Pump%20Instructions%20and%20Repair.pdf

[13] Ecologix Systems, Wastewater Plant Troubleshooting Guide: Pumps, https://ecologixsystems.com/articles/wastewater-pump-troubleshooting

[14] Oxmaint, Centrifugal Pump Failures & Troubleshooting Guide for Manufacturing Plants, https://oxmaint.com/industries/manufacturing-plant/centrifugal-pump-common-failures

[15] HISCO Pump, How Do You Troubleshoot a Centrifugal Pump?, https://hiscopump.com/how-do-you-troubleshoot-a-centrifugal-pump

[16] Nuova Darimpianti, Centrifugal Pump Troubleshooting: Cavitation, Vibration and Seal Leaks, 2026, https://nuovadarimpianti.it/en/guides-and-insights/pumps/centrifugal-pump-troubleshooting-guide/

[17] Chem-Casts, A Practical Guide to Troubleshooting Centrifugal Pump Cavitation, https://chem-casts.com/knowledge/a-practical-guide-to-troubleshooting-centrifugal-pump-cavitation

[18] StepperOnline, "Gearbox - Troubleshooting Guide," updated 2026, https://help.omc-stepperonline.com/hc/s/articles/gearbox-troubleshooting-guide

[19] Malloy Electric, "Industrial Gearbox Troubleshooting Guide," n.d., https://www.malloyelectric.com/gearbox-troubleshooting

[20] Ergenc, "Gearbox: troubleshooting step by step," updated 2026, https://www.ergenc.com/en/kennis/maintenance/gearbox-troubleshooting-step-by-step/

[21] Cotta, "Technical Information and Resources – Gearbox Overheating," n.d., https://cotta.com/downloads-online-forms/technical-information-faq/

[22] Ho Hsing Precision Industry Co., Ltd., "Gearbox Operation Manual," circa 2018, https://www.k-online.de/vis-content/event-k2019/exh-k2019.2626823/K-2019-Ho-Hsing-Precision-Industry-Co.-Ltd.-Product-k2019.2626823-PWxD5DoNSfCiO7cs8gQu1A.pdf

[23] Fabrico, Air Compressor Troubleshooting: Common Problems, Causes, and Fixes, 2026, https://www.fabrico.io/blog/air-compressor-troubleshooting/

[24] Ceccato, Air Compressor Troubleshooting Guide: solve common problems, n.d., https://www.ceccato.com/en-au/blog/air-compressor-troubleshooting-guide

[25] Fabrico, Rotary Screw Compressor Maintenance: Oil, Air-End and Bearings, 2026, https://www.fabrico.io/blog/screw-compressor-maintenance/

[26] afumetto, Predictive Maintenance Dataset (Air Compressor), Kaggle, n.d., https://www.kaggle.com/datasets/afumetto/predictive-maintenance-dataset-air-compressor

[27] IIT Kanpur IDEA Lab, Air Compressor Dataset Repository, n.d., https://iitk.ac.in/idea/datasets/index.html